# UD 5

# Principios de la POO

–

## Módulo de Programación
## 1º DAW

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

Autor: Fran Gómez

2025/2026

# Objetivos

- **Entender** las principales propiedades de la POO
- **Aplicar** estas propiedades en las aplicaciones usando Java
- **Mejorar** destreza en el uso de Java

# Índice de contenidos resumido



1. Encapsulación (visibilidad)
2. Clases y miembros Estáticos
3. Paquetes
4. Librerías
5. Herencia
6. Polimorfismo
7. Clase Object
8. Clases y métodos Abstractos
9. Clases y miembros Finales
10. Composición
11. Paso por valor y por referencia

# ¿Qué se va a evaluar?



| | Mod. | RA |
|---|---|---|
| **RA 4 Desarrolla programas organizados en clases analizando y aplicando los principios de la programación orientada a objetos.** | 10% | |
| a) Se ha reconocido la sintaxis, estructura y componentes típicos de una clase. | 1% | 10% |
| b) Se han definido clases. | 1% | 10% |
| c) Se han definido propiedades y métodos. | 1% | 10% |
| d) Se han creado constructores. | 1% | 10% |
| e) Se han desarrollado programas que instancien y utilicen objetos de las clases creadas anteriormente. | 1% | 10% |
| f) Se han utilizado mecanismos para controlar la visibilidad de las clases y de sus miembros. | 1% | 10% |
| g) Se han definido y utilizado clases heredadas. | 1% | 10% |
| h) Se han creado y utilizado métodos estáticos. | 1% | 10% |
| j) Se han creado y utilizado conjuntos y librerías de clases. | 1% | 10% |

# ¿Qué se va a evaluar?



| | Mod. | RA |
|---|---|---|
| **RA 7 Desarrolla programas aplicando características avanzadas de los lenguajes orientados a objetos y del entorno de programación.** | 20% | |
| a) Se han identificado los conceptos de herencia, superclase y subclase. | 4% | 20% |
| b) Se han utilizado modificadores para bloquear y forzar la herencia de clases y métodos. | 4% | 20% |
| c) Se ha reconocido la incidencia de los constructores en la herencia. | 2% | 10% |
| d) Se han creado clases heredadas que sobrescriban la implementación de métodos de la superclase. | 2% | 10% |
| e) Se han diseñado y aplicado jerarquías de clases. | 2% | 10% |
| f) Se han probado y depurado las jerarquías de clases. | 2% | 10% |
| g) Se han realizado programas que implementen y utilicen jerarquías de clases. | 2% | 10% |
| h) Se ha comentado y documentado el código. | 2% | 10% |

# ¿Cómo lo vamos a evaluar?



| Instrumento | Técnica |
|---|---|
| **Portfolio**:<br>- Se evalúa el trabajo diario<br>- Recurso para estudiar<br>- Útil como CV | - Salir a corregir los ejercicios de clase<br>- Defender los ejercicios necesarios frente al profesor antes del examen. |
| **Proyecto**:<br>- Evaluación de todo lo aprendido y más<br>- Aplicación en diferentes contextos<br>- Colaborativo | - Exposición: explicar qué y cómo funciona a la clase, respondiendo a las preguntas. |
| **Examen**:<br>- Evaluación individual y sumativa de lo fundamental | - Prueba escrita |

# Encapsulación



La **encapsulación** es un principio de la Programación Orientada a Objetos que consiste en:

- **Ocultar los datos internos** de una clase.
- **Controlar el acceso** a esos datos mediante métodos.
- Proteger el estado del objeto y evitar usos incorrectos.

*Los atributos no se tocan directamente desde fuera de la clase.*

# Encapsulación

## Problema sin encapsulación (ejemplo)

Declaración de una clase:

```
class Persona {
    public String nombre;
    public int edad;
}
```

```
class Persona {
    private String nombre;
    private int edad;
}
```

Uso:

```
Persona p = new Persona();
p.edad = -5;  // ❌ No tiene sentido
```

**Reflexión:**

- ¿Es válida una edad negativa?
- ¿Quién controla los datos?
- ¿Dónde está el error de diseño?

# Encapsulación



Solución:

1. Atributos privados
2. Métodos getter y setter

```
Persona p = new Persona();
p.setNombre("Ana");
p.setEdad(20);
```

Ya no se puede hacer `p.edad = -5`.

El control está dentro de la clase.

```
class Persona {
    private String nombre;
    private int edad;

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public int getEdad() {
        return edad;
    }

    public void setEdad(int edad) {
        if (edad >= 0) {
            this.edad = edad;
        }
    }
}
```

## Beneficios de la encapsulación

- Protege los datos del objeto.
- Evita errores difíciles de detectar.
- Facilita el mantenimiento del código.
- Permite cambiar la implementación interna sin afectar al resto del programa.

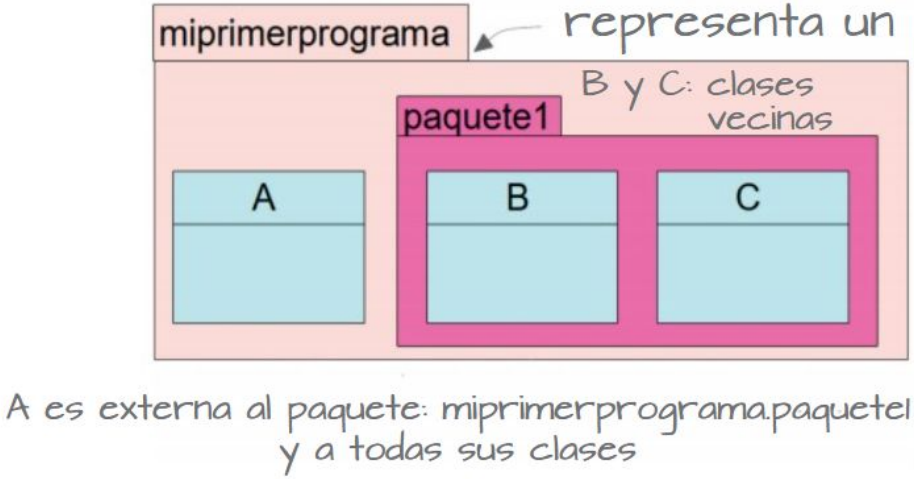
# Encapsulación: visibilidad

La visibilidad nos dice si algo podemos usarlo o no:

- Es visible → podemos usarlo
- No es visible → no podemos usarlo

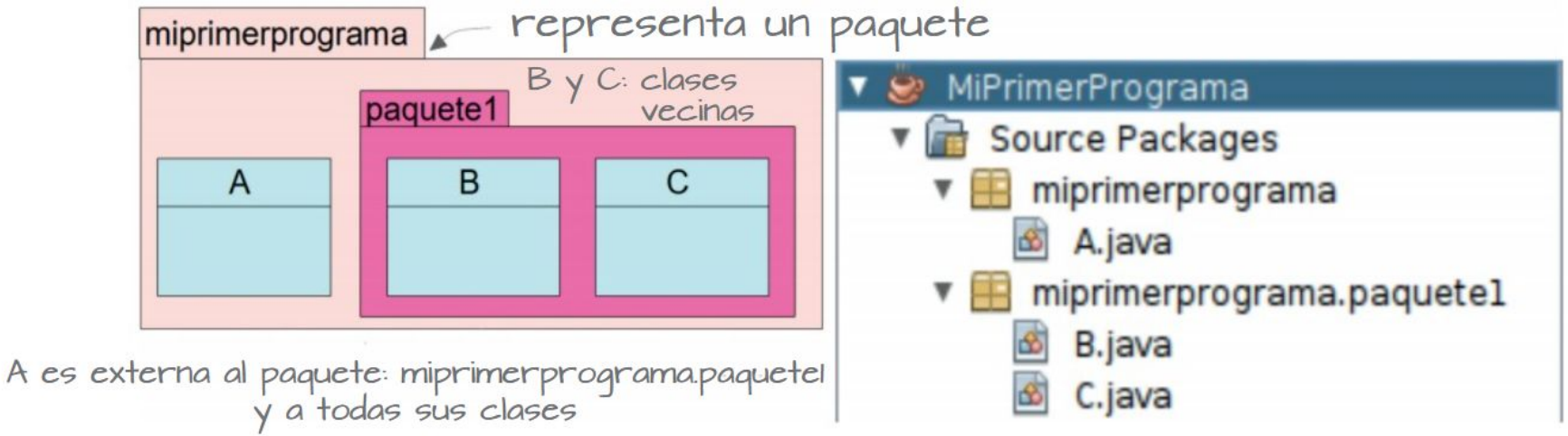
La visibilidad se controla de dos formas:

- Paquetes
- Modificadores de visibilidad



# Visibilidad a nivel de clase

Por defecto, si se ubican en el mismo paquete pueden verse



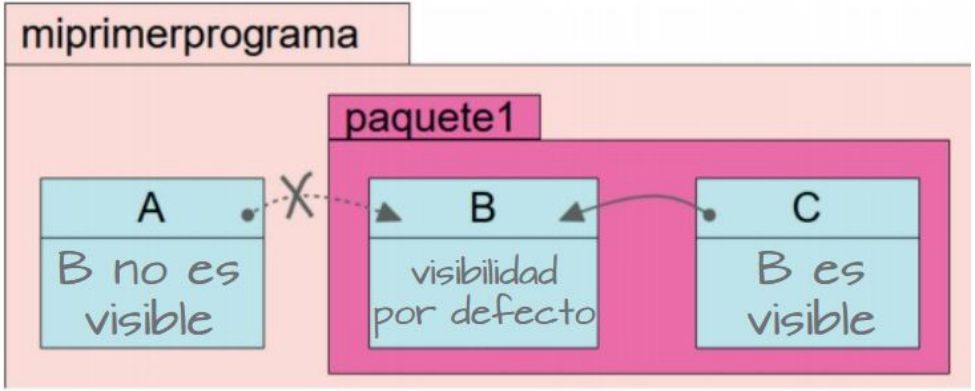
# Visibilidad a nivel de clase

Usando modificadores al declarar la clase:

```
[visibilidad] class NombreClase {
    ...
}
```

- **public** → clase visible para todas las clases
- **package** (implícito si no se pone nada) → clase visible para todas las clases del mismo paquete

```
package miprimerprograma.paquete1;
class B { //sin modificador de acceso
    ...
}
```




**Buena práctica**: visibilidad lo más restrictiva posible

# Visibilidad a nivel de clase



```
package miprimerprograma.paquete1;
public class B { //clase marcada como pública
    ...
}

package miprimerprograma;
import miprimerprograma.paquete1.B; //ahora A puede usar la clase B

class A {
    ...
}
```

**Buena práctica**: visibilidad lo más restrictiva posible

# Visibilidad a nivel de clase

Se importan las clases no los paquetes.

Para importar todas las clases de un paquete → *

```
package miprimerprograma;
import miprimerprograma.paquete1.*; //A puede usar cualquier clase pública
                                    //del paquete miprimerprograma.paquete1
class A {
    ...
}
```

| | Visible desde… | |
|---|---|---|
| | clases vecinas | clases externas |
| sin modificador | ✓ | |
| public | ✓ | ✓ |

**Buena práctica**: visibilidad lo más restrictiva posible

# Visibilidad a nivel de miembros



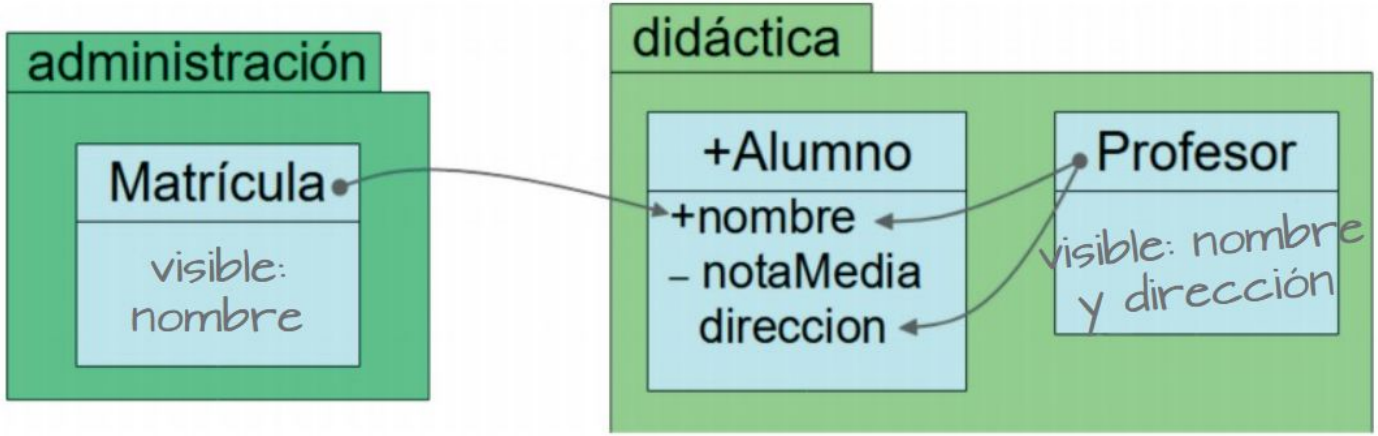
A nivel de los miembros de una clase:

| | | Mismo paquete | | Otro paquete | |
|---|---|---|---|---|---|
| | | Subclase | Otra | Subclase | Otra |
| - | **private** | *no* | *no* | *no* | *no* |
| # | **protected** | *sí* | *sí* | *sí* | *no* |
| + | **public** | *sí* | *sí* | *sí* | *sí* |
| ~ | *package* | *sí* | *sí* | *no* | *no* |

**Buena práctica**: visibilidad lo más restrictiva posible

# Visibilidad a nivel de miembros






+ significa público

y

- significa privado

# Encapsulación: getter y setter



La encapsulación permite ocultar detalles de la implementación y controlar qué atributos pueden usarse libremente o incluso cómo deben usarse.

Para conseguir esto, hay que combinar la visibilidad con métodos que acceden a los atributos para leerlos y escribirlos.

# Encapsulación: getter y setter



Estos métodos se identifican con set/get seguido del nombre del atributo. Para el atributo `edad` quedaría:

```
class Persona {
   private int edad;
   ...

   public void setEdad(int edad) {
      if (edad >= 0) //solo los valores positivos tienen sentido
         this.edad = edad;
      } // en caso contrario no se modifica la edad
   }

   public int getEdad() {
      return edad;
   }
}
```

# Encapsulación: Ejercicio 1



Cree una clase `CuentaBancaria` que cumpla lo siguiente:

- Atributos privados:
  - `titular`
  - `saldo`
- Métodos públicos:
  - `getTitular()`
  - `getSaldo()`
  - `ingresar(double cantidad)` (solo si es positiva)
  - `retirar(double cantidad)` (solo si hay saldo suficiente)

# Ejercicio 2: Mejorando la clase Persona



Copia la clase Persona del tema anterior y ampliala con lo siguiente:

1. Establece todos los atributos de la clase persona para que sólo sean visibles desde la misma clase.
2. Intenta modificar el nombre de una persona desde la clase Ciudad.
3. Para permitir lo anterior, permite que sólo el nombre sea visible desde cualquier clase.
4. Pon todos los atributos con una visibilidad que sólo puedan ser accedidos fuera de la clase, mediante métodos get y set.
5. Genera los correspondientes getter y setter para todos los atributos
6. Añade varias validaciones mediante los setter

# Ejercicio 2 (continúa)



¿Es correcto este código?

```java
package modelo;

public class Persona {
    String nombre; // visibilidad por defecto
}

package app;

import modelo.Persona;
public class Main {
    public static void main(String[] args) {
        Persona p = new Persona();
        p.nombre = "Ana";
    }
}
```

# Ejercicio 3: Test de autoevaluación



Join this Wooclap event

1 Go to wooclap.com

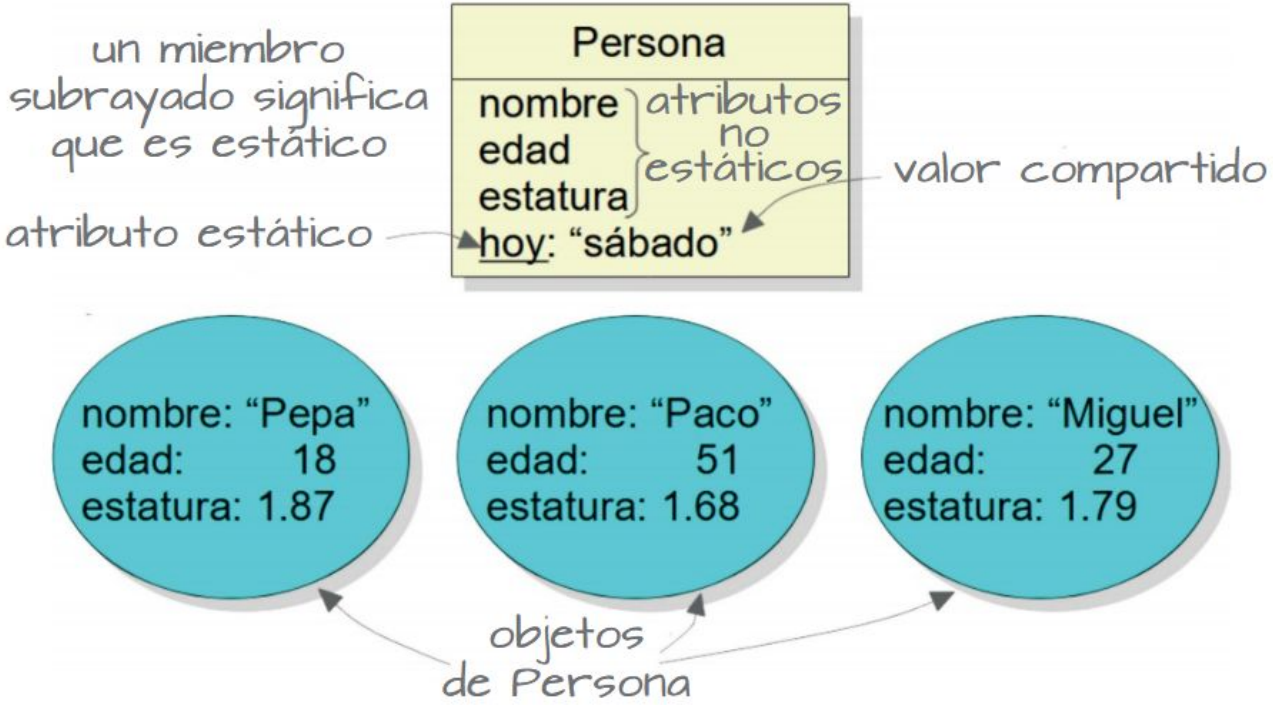
2 Enter the event code in the top banner

Event code
**LCZRFR**

Enable answers by SMS

# Atributos y métodos estáticos

No pertenecen a los objetos sino a la clase y por tanto son compartidos por todos los objetos de la clase



# Atributos estáticos



Un atributo estático se declara mediante la palabra reservada `static`.

```
class Persona {
    ...
    static String hoy;
}
```

```
Persona.hoy = "domingo";
System.out.println(Persona.hoy);
```

# Métodos estáticos



Pertenecen a la clase, por tanto, se ejecutan sin necesidad de crear ningún objeto → por eso desde un método estático sólo se pueden invocar otros métodos estáticos como pasa en el método main.

```
static void hoyEs(int dia) {
    hoy = switch (dia) {
        case 1-> "lunes";
        case 2-> "martes";
        ...
        case 7-> "domingo";
    }
}
```

```
Persona.hoyEs(2);
```

# Método main



```java
public static void main(String[] args)
{
    System.out.println("Hola");
}
```

¿Qué ocurre si no es `public`?

```
Error: Main method not found in class ...
```

**Modificadores, parámetros y retorno**

- `public` → accesible desde la JVM
- `static` → no necesita un objeto para ejecutarse
- `void` → no devuelve ningún valor
- `String[] args` → permite recibir argumentos desde la línea de comandos

*El método `main` es público porque Java lo busca desde fuera de la clase para empezar el programa.*

# Ejercicio 4



Vamos a implementar una clase llamada ContadorObjetos que tenga un atributo estático para llevar la cuenta del número total de objetos creados de esta clase. Además, tendrá un método estático para mostrar cuántos objetos se han creado hasta el momento.

**Instrucciones:**

1. Crea una clase llamada `ContadorObjetos`.
2. Define un atributo estático llamado `totalObjetos` que sea de tipo entero e inicialízalo en 0.
3. Crea un constructor para la clase que incremente el valor de `totalObjetos` cada vez que se cree un objeto de esta clase.
4. Añade un método estático llamado `mostrarTotalObjetos` que imprima en consola el número total de objetos creados.
5. En el método `main` de otra clase, crea varios objetos de la clase `ContadorObjetos` y utiliza el método estático para mostrar cuántos objetos se han creado.

# Estáticos: Resumen.

## Cuándo usar `static`

- Usa **atributos estáticos** cuando:
  - El dato es común a todos los objetos
  - Ejemplo: contadores, constantes, configuraciones
- Usa **métodos estáticos** cuando:
  - El método no depende de datos de un objeto
  - Ejemplo: utilidades, cálculos, validaciones

**Lo estático pertenece a la clase, no a los objetos.**

- Los **atributos no estáticos** pertenecen a **cada objeto**
- Los **atributos estáticos** pertenecen a **la clase en sí**
- Los **métodos estáticos**:
  - No necesitan objetos para ejecutarse
  - No pueden acceder directamente a miembros no estáticos

# Ejercicio 5: Test de autoevaluación



Join this Wooclap event

1 Go to wooclap.com

2 Enter the event code in the top banner

Event code
**LCZRFR**

Enable answers by SMS

# Paquetes



¿Cómo organizar los ficheros de código fuente?

**Paquete** → conjunto de clases e interfaces relacionadas entre sí

Indica a qué paquete pertenece la clase → ***package*** *identificadordelPaquete;*

Importar una clase de otro paquete→ ***import*** *identificadordelPaquete;*

El nombre de un paquete puede estar **cualificado** (subpaquetes separados por un punto)

Siempre que la clase que queramos usar esté en **otro** paquete ⇒ hay que importarla

No confundir **Visibilidad** con **Importación**

# Paquetes



| *Package* | Descripción del contenido |
|---|---|
| `java.lang` | Contiene las clases e interfaces más empleadas en la mayoría de los programas de Java. Es importado automáticamente por todos los programa Java: no se necesita sentencia `import` para utilizar lo declarado en este paquete. |
| `java.io` | Contiene clases que permiten las operaciones de entrada y salida de datos de un programa. |
| `java.util` | Contiene clases e interfaces de utilidades: operaciones con la fecha y la hora, generación de números aleatorios... |
| `java.applet` | Contiene todas las clases e interfaces necesarias para la construcción de *applets* de Java |
| `java.net` | Contiene clases que permite a un programa comunicarse a traves de redes (Internet o intranet) |
| `java.text` | Contiene clases e interfaces que permiten operaciones de números, fechas, caracteres y cadenas. |
| `java.awt` | Es el paquete *Abstract Windowing Toolkit*. Contiene muchas clases e interfaces necesarias para trabajar con la interfaz de usuario gráfica *clásica*. |
| `java.beans` | Contiene clases para facilitar a los programadores la generación de componentes de software reutilizables. |

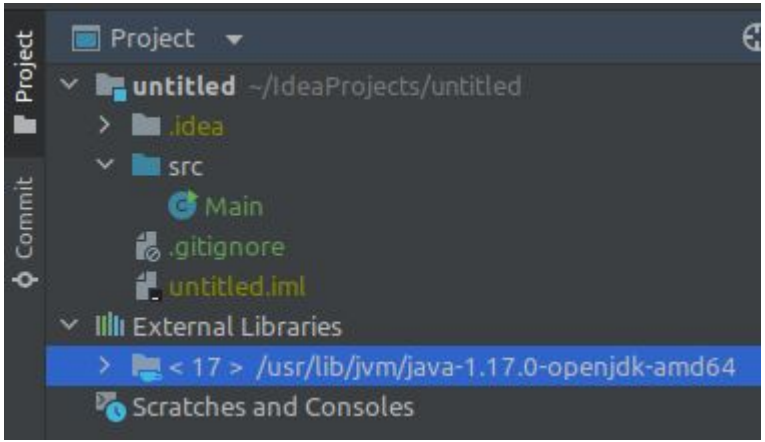
# Librería (o biblioteca)

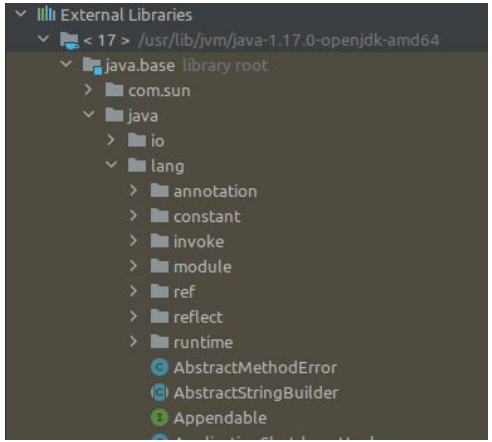


- Es el código ya desarrollado que vamos a utilizar para desarrollar nuevo código.
- En lugar de hacerlo todo desde 0, nos basamos en lo que ya hay hecho. (No reinventamos la rueda)
- De esta forma aceleramos el desarrollo, evitamos errores, facilitamos el mantenimiento, etc… algo parecido a lo que obteníamos organizando el código en funciones, pero a mayor escala.
- Las librerías se distribuyen encapsuladas en uno o varios ficheros. En el caso de java tenemos los **.jar**

# Librería (o biblioteca)

- Tanto java.Math como java.util.Ramdon son clases (que están organizadas en diferentes paquetes) de la librería de Java: jdk



# Librería (o biblioteca) - Ejercicio 6



Busca las siguientes clases dentro de tu librería JDK:

- Object
- String
- Math
- java.util.Random

# Librería (o biblioteca)



¿Puedo crear mi propia librería?

Podemos hacerlo y compartir nuestros desarrollos con los demás.

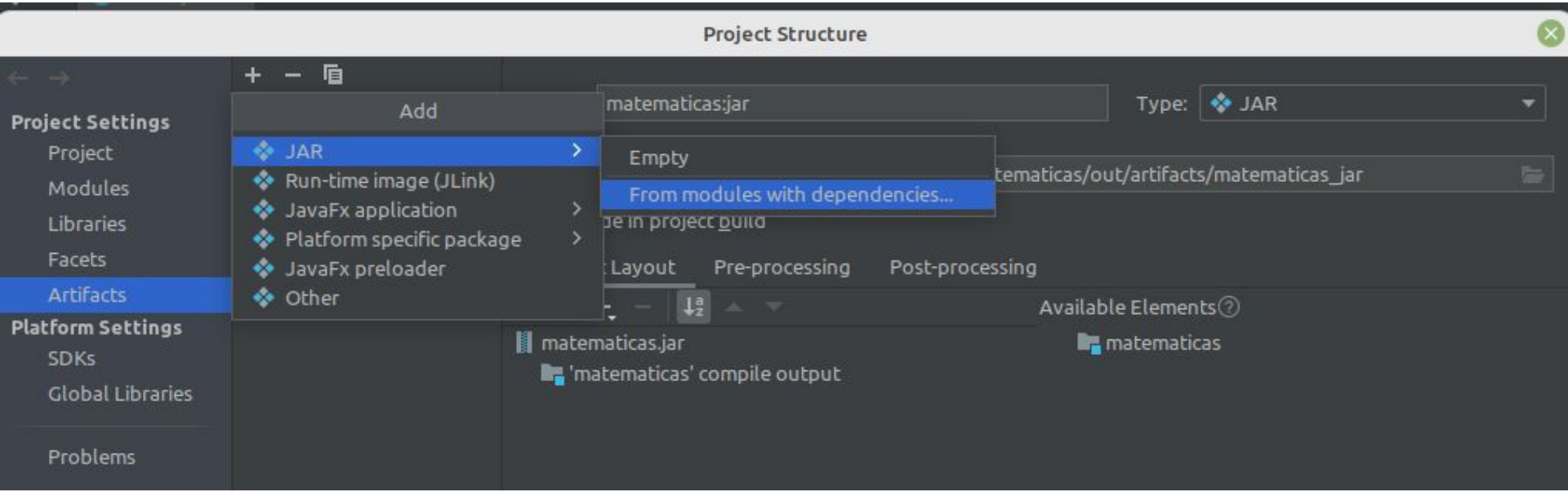
Veamos cómo…

# Librería (o biblioteca)



1. Creamos un proyecto nuevo con las clases de nuestra librería

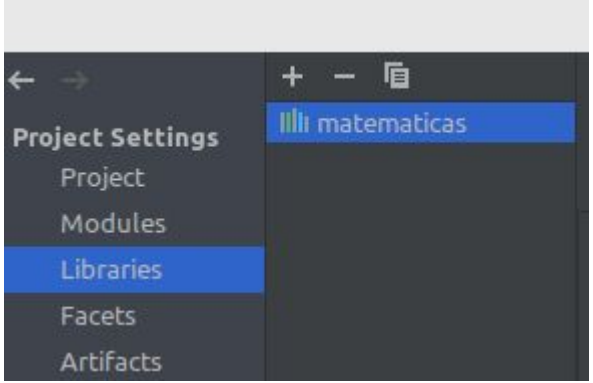
2. Generamos un .jar

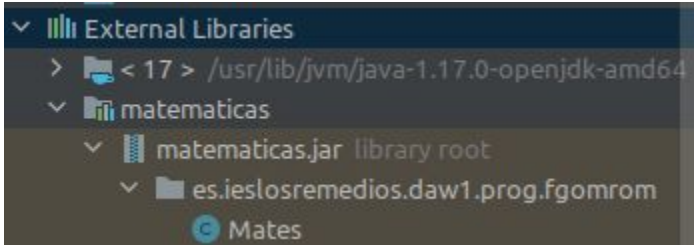



3. Añadimos la librería (el .jar generado) a un segundo proyecto en el que queramos hacer uso de ella.

# Librería (o biblioteca)





# Librerías: Ejercicio 6



Vamos a usar nuestra clase CalculadoraBasica dentro de otro proyecto.

Lo primero es convertir sus métodos y atributos en estáticos, ya que realmente es una **clase de utilidades**, como java.lang.Math.

Por otro lado, hay que asegurarse de que los métodos son públicos, pues de otra forma no podremos usarlos desde otros paquetes.

# Librerías: Ejercicio 6



Seguidamente generamos un **jar** para empaquetar y distribuir nuestra clase CalculadoraBasica.

**Nota**: un jar puede ser ejecutable o no, pues también es la forma de empaquetar una aplicación en Java. En nuestro caso, al ser una librería, no tiene que ser ejecutable.

Por último añadimos la librería en un **nuevo proyecto** donde queramos usarla.

# Relaciones entre clases

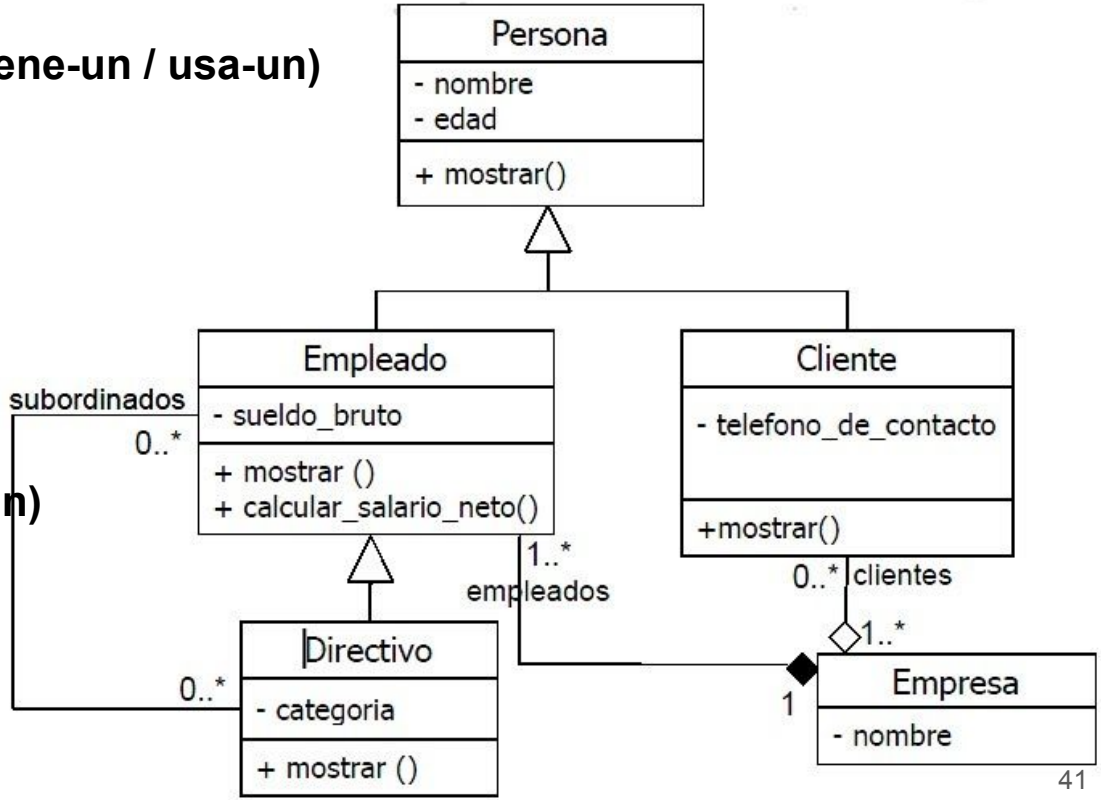
En Programación Orientada a Objetos, las relaciones entre clases pueden agruparse en dos grandes bloques:

## A. Relaciones de *uso o pertenencia* (tiene-un / usa-un)

- Asociación
- Agregación
- Composición

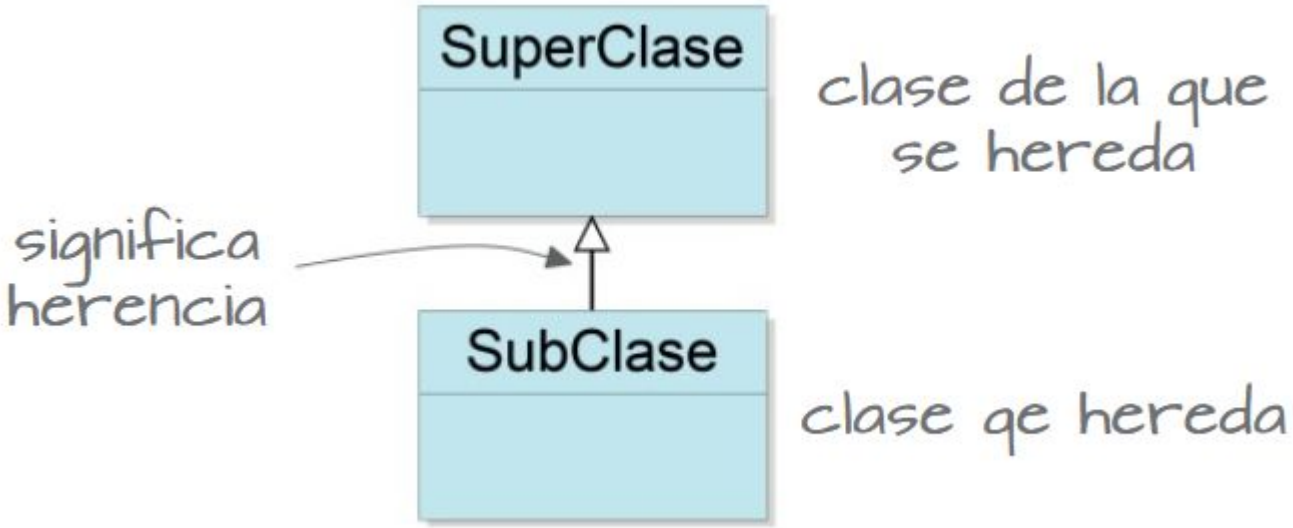
## B. Relaciones de *especialización* (es-un)

- Herencia



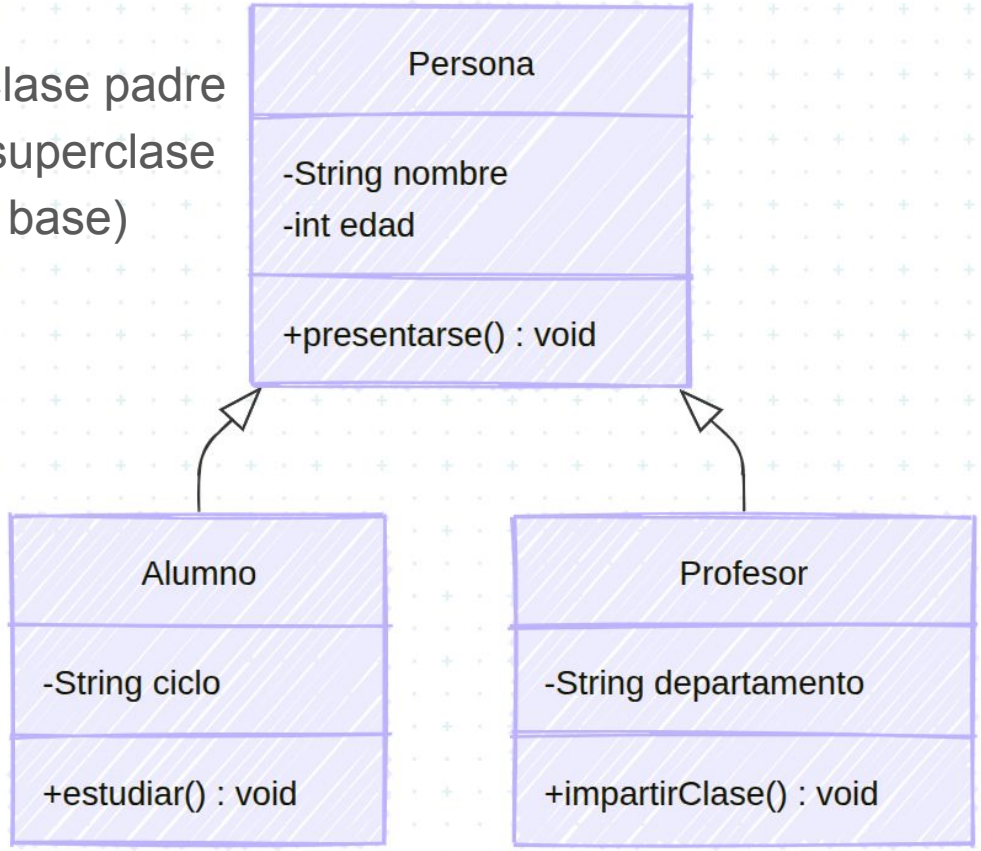
# Herencia

**<u>Objetivo</u>: crear una clase nueva a partir de otra existente**, reutilizando su código y ampliándolo o modificándolo.



# Herencia

Clase padre (superclase o base)




- Es una especialización (relación “es un”). Ej:
  - Alumno **es una** Persona
  - Profesor **es una** Persona

- Todos los miembros de la superclase son heredados por la subclase (excepto constructores)

y clases hijas (subclases o derivadas)

# Herencia: sintaxis y ejemplo

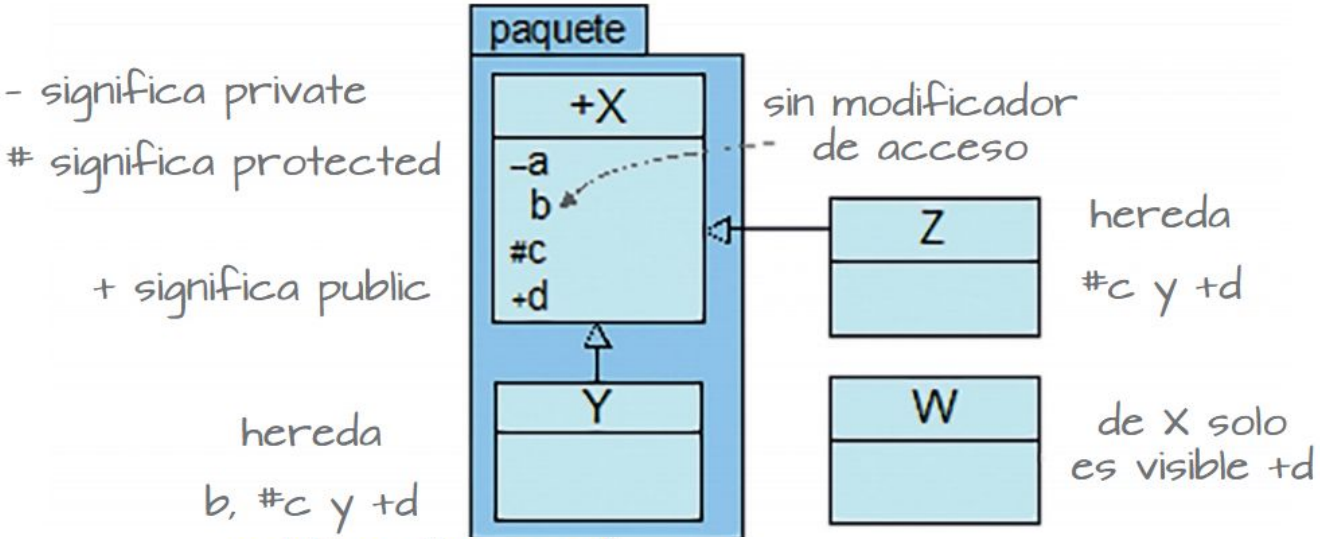


```
class SubClase extends SuperClase {

    ...

}
```

```java
public class Persona {
    protected String nombre;
    protected int edad;

    public void presentarse() {
        System.out.println("Hola, me llamo " + nombre);
    }
}
```

```java
public class Alumno extends Persona {
    private String ciclo;

    public void estudiar() {
        System.out.println("Estoy estudiando " + ciclo);
    }
}
```

# Herencia: visibilidad




```
public class X {
        private int a; //invisible fuera de la clase
        int b; //visibilidad por defecto: visible en el paquete
        protected int c; //visible en el paquete y para
        //las subclases (aunque sean externas)
        public int d; //visibilidad total
}
```

# Herencia: visibilidad



| | Visible desde... | | | |
|---|---|---|---|---|
| | la propia clase | clases vecinas | subclases | clases externas |
| `private` | ✓ | | | |
| *sin modificador* | ✓ | ✓ | | |
| `protected` | ✓ | ✓ | ✓ | |
| `public` | ✓ | ✓ | ✓ | ✓ |

# Herencia: Ejemplo



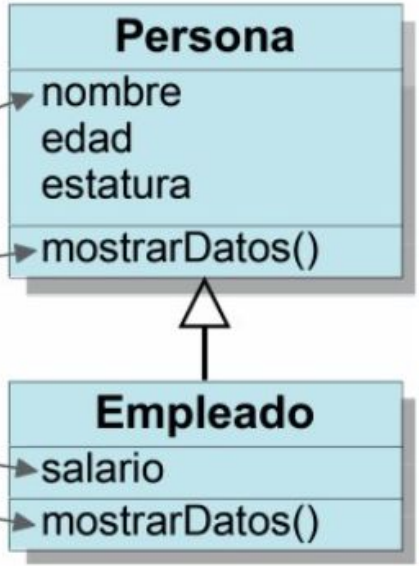
```
class Persona {
    String nombre;
    byte edad;
    double estatura;
}
class Empleado extends Persona {
    double salario;
    Empleado(String nombre, byte edad, double estatura, double salario) {
        ...
    }
}
```

```
Empleado e = new Empleado("Sancho",  25,  1.80,  1725.49);
System.out.println(e.nombre); //muestra un atributo heredado
System.out.println(e.salario); //muestra un atributo propio
```

# Herencia: Override



```
Persona p = new Persona();
Empleado e = new Empleado();

e.nombre
p.mostrarDatos()
e.salario
e.mostrarDatos()
```




Un miembro redefinido en la subclase estará sobreescribiendo el miembro heredado de la superclase

- Subclase declara mismo método que la superclase (mismo nombre, parámetros y tipos)
- Añadimos funcionalidad en la subclase sobre la que ofrecía la superclase
- Anotación @Override

# Herencia: Override



```
class Persona {
    String nombre;
    byte edad;
    double estatura;
    void mostrarDatos() {
        System.out.println(nombre);
        System.out.println(edad);
        System.out.println(estatura);
    }
}
class Empleado extends Persona {
    double salario;
    @Override //significa: sustituye un método de la superclase
    void mostrarDatos() {
        System.out.println(nombre);
        System.out.println(edad);
        System.out.println(estatura);
        System.out.println(salario);
    }
}
```

# Herencia: Override



```java
public class Persona {
    protected String nombre;

    public void presentarse() {
        System.out.println("Soy una persona");
    }
}
```

```java
public class Alumno extends Persona {

    @Override
    public void presentarse() {
        System.out.println("Soy un alumno");
    }
}
```

# Herencia: super y super()



```
public class Persona {
     String nombre;
     byte edad;
     double estatura;
     ...
     void mostrarDatos() {
            System.out.println(nombre);
            System.out.println(edad);
            System.out.println(estatura);
     }
}
class Empleado extends Persona {
     double salario;
     @Override
     void mostrarDatos() {
            super.mostrarDatos(); /*método de la superclase, muestra los
            atributos definidos en Persona*/
            System.out.println(salario); /*muestra el atributo añadido en
            Empleado*/
     }
}
```

super permite **reutilizar y extender.**

```
public class Alumno extends Persona {

    @Override
    public void presentarse() {
        super.presentarse();
        System.out.println("y estudio en el centro");
    }
}
```

# Herencia: super y super()



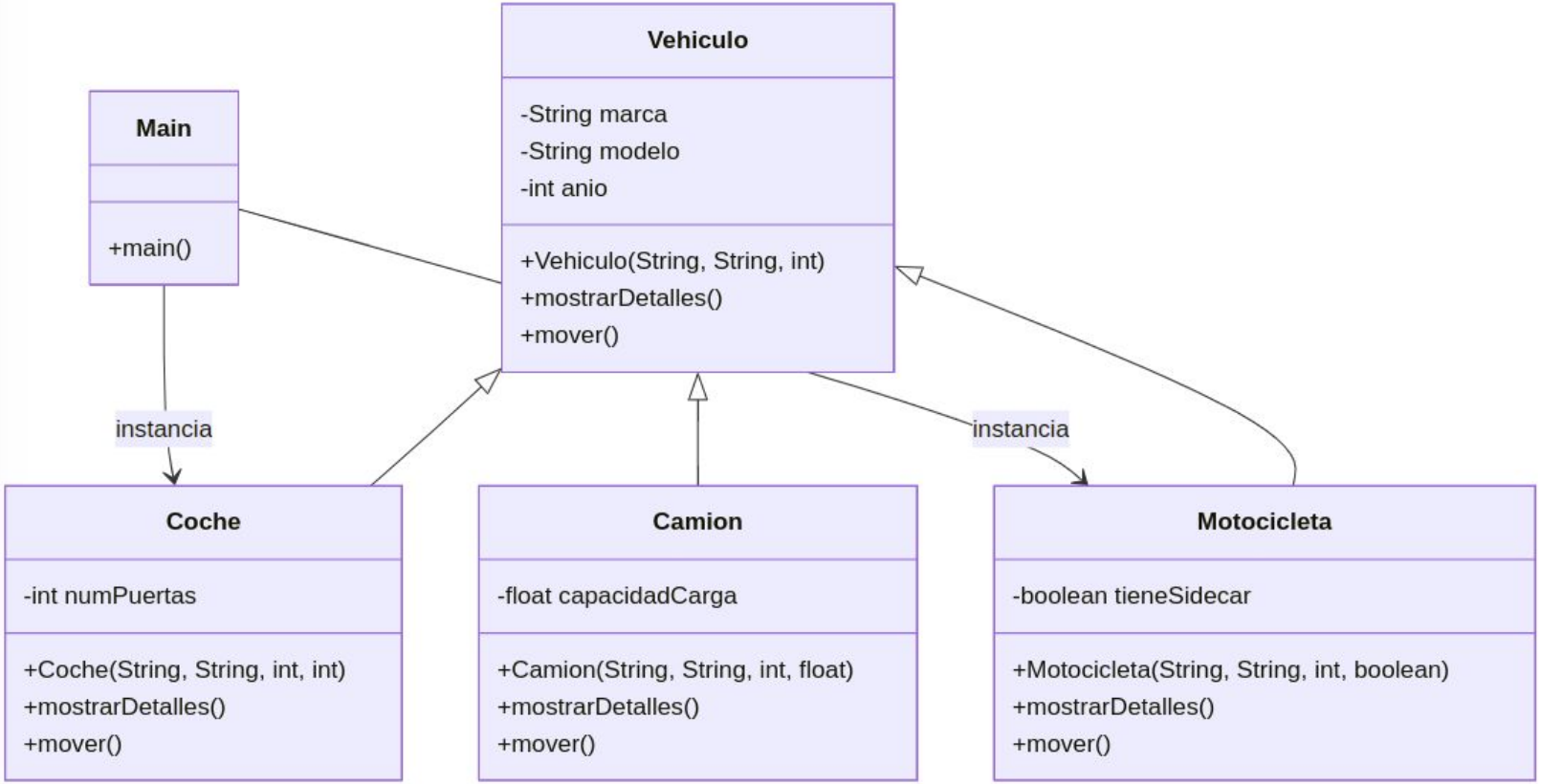
```java
public class Persona {
    protected String nombre;

    public Persona(String nombre) {
        this.nombre = nombre;
    }
}
```

```java
public class Alumno extends Persona {
    private String ciclo;

    public Alumno(String nombre, String ciclo) {
        super(nombre);
        this.ciclo = ciclo;
    }
}
```

`super()` permite **reutilizar y extender** el constructor

# Ejercicio 7




Vehiculo
-String marca
-String modelo
-int anio
+Vehiculo(String, String, int)
+mostrarDetalles()
+mover()
Main
+main()
instancia
instancia
Coche
-int numPuertas
+Coche(String, String, int, int)
+mostrarDetalles()
+mover()
Camion
-float capacidadCarga
+Camion(String, String, int, float)
+mostrarDetalles()
+mover()
Motocicleta
-boolean tieneSidecar
+Motocicleta(String, String, int, boolean)
+mostrarDetalles()
+mover()

# Herencia



Otras consideraciones:

- No existe herencia múltiple en Java
- Sí existe herencia transitiva
- La primera clase en la jerarquía es Object

# Herencia

Join this Wooclap event

1 Go to wooclap.com

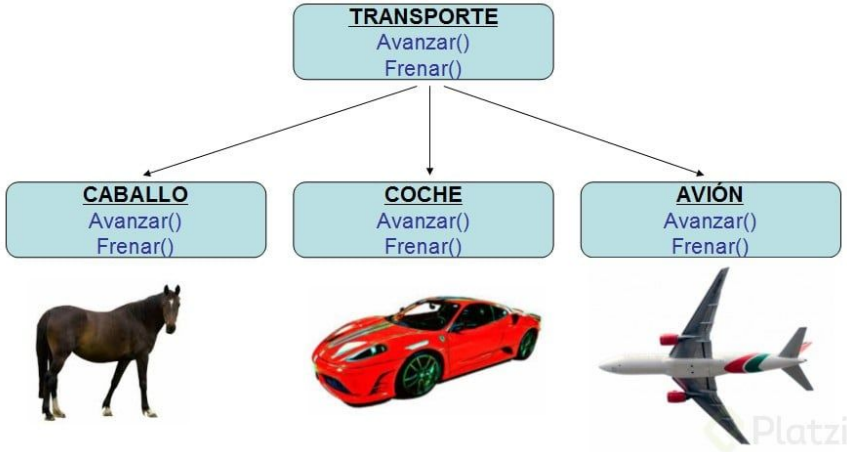
2 Enter the event code in the top banner

Event code
FJNLCX

Enable answers by SMS

Copy participation link

# Polimorfismo



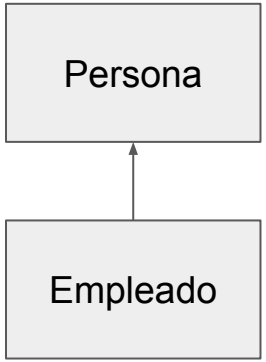

El **polimorfismo** es un principio de la Programación Orientada a Objetos que permite que **una misma referencia** se comporte de **distintas formas** según el objeto concreto al que apunte.

En Java, esto significa que:

- Una variable de tipo **clase padre** puede referenciar objetos de **clases hijas**.
- El **método que se ejecuta** se decide en tiempo de ejecución, no en compilación.

Un objeto se comporta según su tipo real, no según el tipo de la variable que lo referencia.

```
Empleado e = new Empleado();

Persona p = e;
```



# Polimorfismo



```java
public class Animal {
    public void hacerSonido() {
        System.out.println("El animal hace un sonido");
    }
}
```

```java
public class Perro extends Animal {
    @Override
    public void hacerSonido() {
        System.out.println("El perro ladra");
    }
}
```

```java
public class Gato extends Animal {
    @Override
    public void hacerSonido() {
        System.out.println("El gato maúlla");
    }
}
```

# Polimorfismo



```
Animal a1 = new Perro();
Animal a2 = new Gato();

a1.hacerSonido(); // El perro ladra
a2.hacerSonido(); // El gato maúlla
```

Aunque `a1` y `a2` son de tipo `Animal`, Java ejecuta el método correspondiente al **objeto real** (`Perro` o `Gato`).

# Polimorfismo



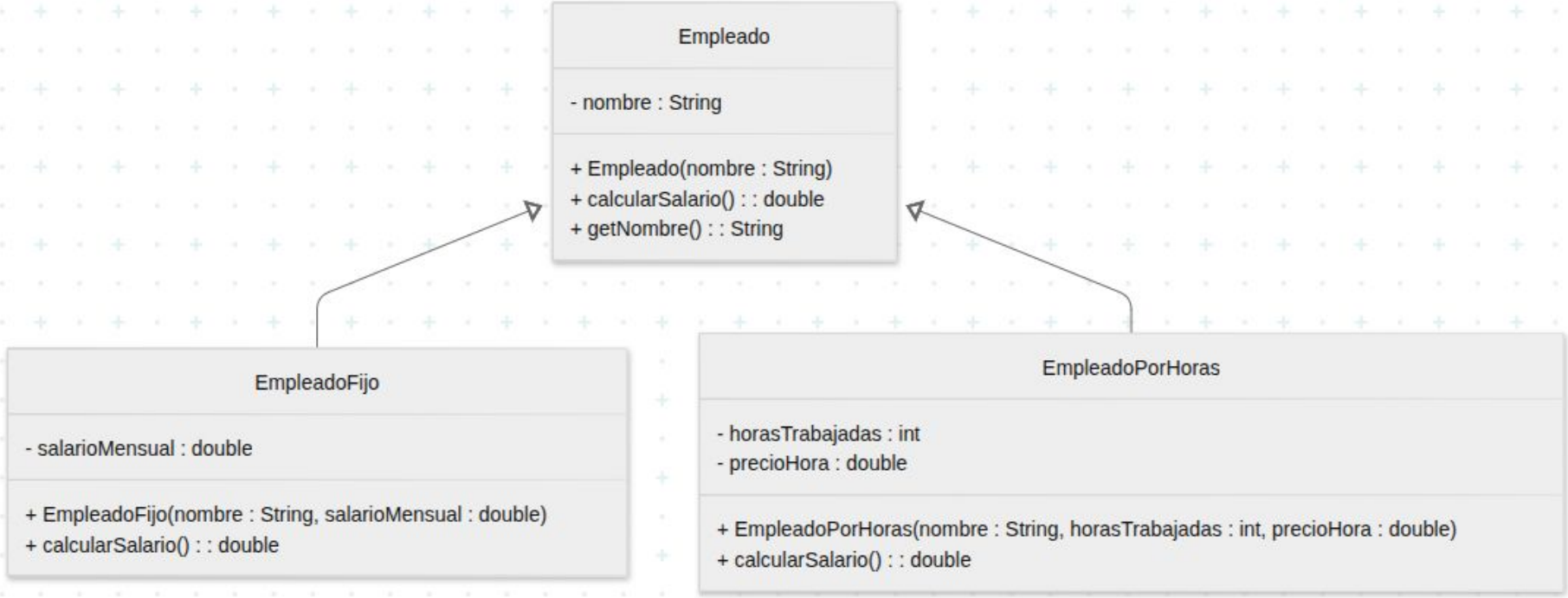
El polimorfismo permite que una referencia de una clase padre invoque métodos redefinidos en sus clases hijas, ejecutándose el comportamiento correspondiente al tipo real del objeto en tiempo de ejecución.

Ventajas de usar polimorfismo:

- Código más **flexible** ya que una referencia puede tomar una forma concreta en tiempo de ejecución.

# Ejercicio 8




Empleado
- nombre : String
+ Empleado(nombre : String)
+ calcularSalario() : : double
+ getNombre() : : String
EmpleadoFijo
- salarioMensual : double
+ EmpleadoFijo(nombre : String, salarioMensual : double)
+ calcularSalario() : : double
EmpleadoPorHoras
- horasTrabajadas : int
- precioHora : double
+ EmpleadoPorHoras(nombre : String, horasTrabajadas : int, precioHora : double)
+ calcularSalario() : : double

# Ejercicio 9



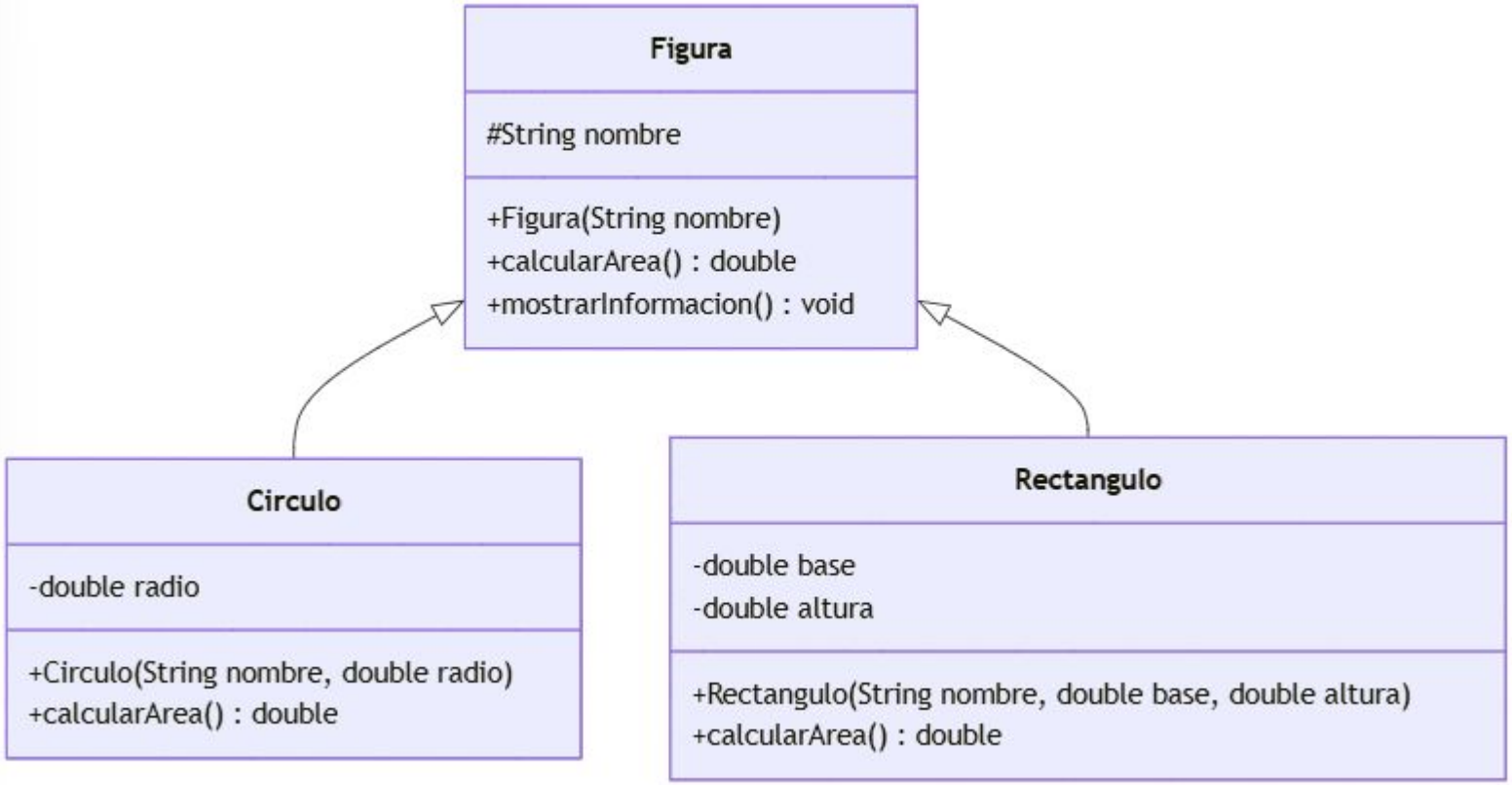


Figura
#String nombre
+Figura(String nombre)
+calcularArea() : double
+mostrarInformacion() : void
Circulo
-double radio
+Circulo(String nombre, double radio)
+calcularArea() : double
Rectangulo
-double base
-double altura
+Rectangulo(String nombre, double base, double altura)
+calcularArea() : double

# Ejercicio 10



Join this Wooclap event

1 Go to wooclap.com

2 Enter the event code in the top banner

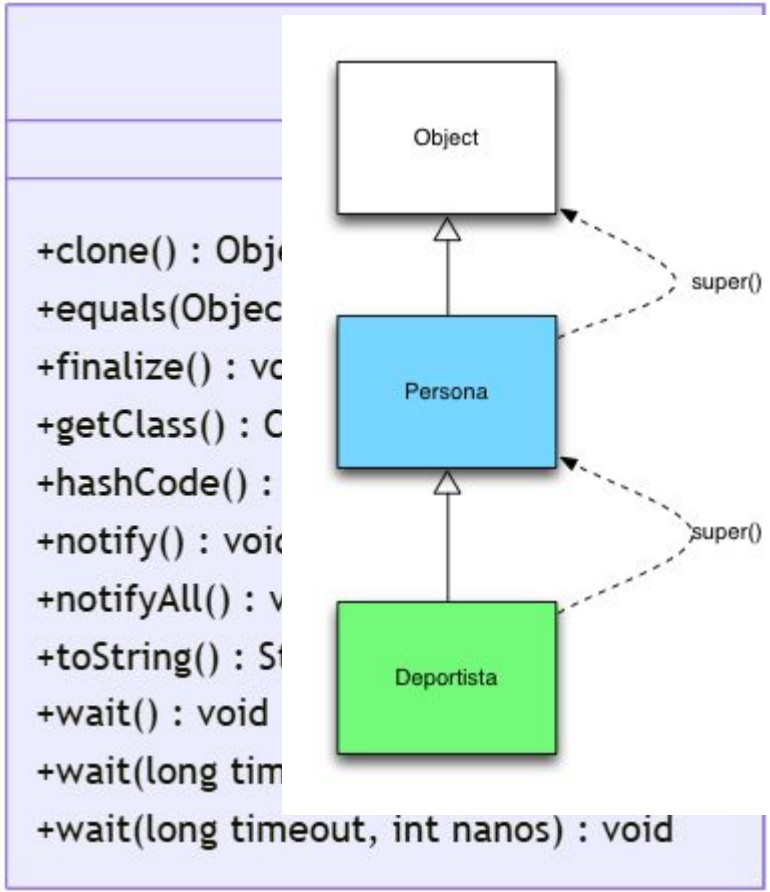
Event code
**FSBXEN**

Enable answers by SMS

# La clase Object



- La clase `Object` es la superclase base de todas las clases en Java.
- Todas las clases en Java heredan implícitamente de `Object`.
- Permite que cualquier variable de tipo `Object` pueda referirse a un objeto de cualquier clase.
- Proporciona funcionalidad esencial y común a todas las clases.
- Métodos como `equals()`, `toString()`, y `hashCode()` son frecuentemente sobrescritos para personalizar el comportamiento.



# Métodos de Object



| Método | Descripción |
|---|---|
| Object clone() | Crea un nuevo objeto, igual al que se duplica |
| boolean equals (Object objeto) | Determina si un objeto es igual a otro |
| void finalize() | Llama antes de que un objeto no utilizado sea reciclado. |
| Class getClass() | Obtiene la clase de un objeto en tiempo de ejecución. |
| int hashCode() | Devuelve el código asociado al objeto que realiza la llamada. |
| void notify() | Reiniciar la ejecución de un hilo en espera en el objeto que realiza la llamada. |
| void notifyAll() | Reiniciar la ejecución de todos los hilos en espera en el objeto que realiza la llamada. |
| String toString() | Devuelve una cadena que describe el objeto |
| void wait()<br>void wait (long milisegons)<br>void wait (long milisegons,int nanosegons) | Espera en otro subproceso de ejecución |

Los métodos ***getClass***, ***notify***, ***notifyAll*** y ***wait*** están declarados como ***final***, el resto pueden sobrescribir.

# toString



¿Qué hace?

- Convierte un objeto en una representación de cadena

¿Para qué se usa?

- Proporciona una representación legible del objeto para poder imprimirlo

Ejemplo de salida

- `Persona@1a2b3c4d`

¿Cómo podríamos utilizarlo para imprimir nuestros objetos personalizados?

```
public class Persona {
    private String nombre;
    private int edad;

    @Override
    public String toString() {
        return "Persona: " + nombre + ", Edad: " + edad;
    }
}
```

# toString

## ¿Cómo lo usamos?

Uso explícito

- `Persona persona = new Persona("Fran", 30);`
  `persona.toString();` → **`Persona: Fran, Edad: 30`**

Uso implícito

- `System.out.println(persona)`
- `String.valueOf(persona)`
- Concatenación de cadenas: `"Info: " + persona`

## Buenas prácticas

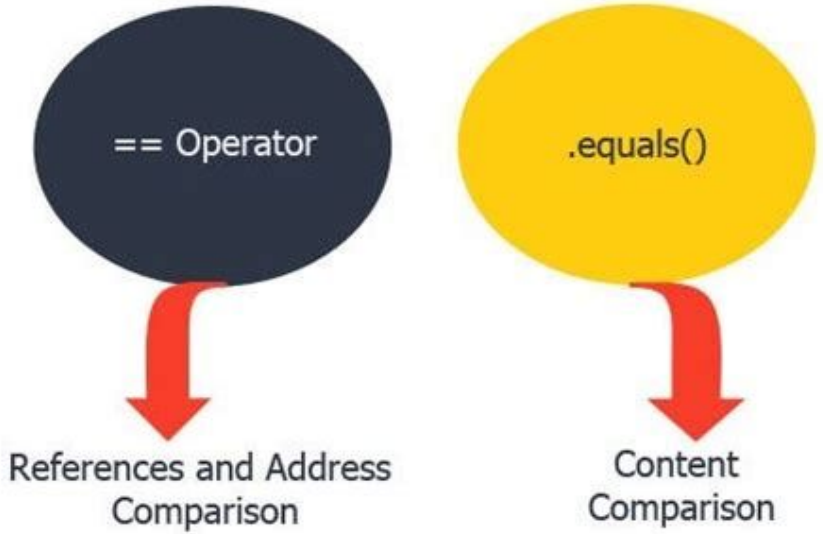
- Sobrescribir en clases propias
- Incluir atributos relevantes
- Mantener un formato consistente

# equals()

Sirve para comparar si dos objetos son **equivalentes en términos de contenido**, no solo si son el mismo objeto en memoria.

Implementación por defecto:

```
public boolean equals(Object obj){
    return (this == obj);
}
```



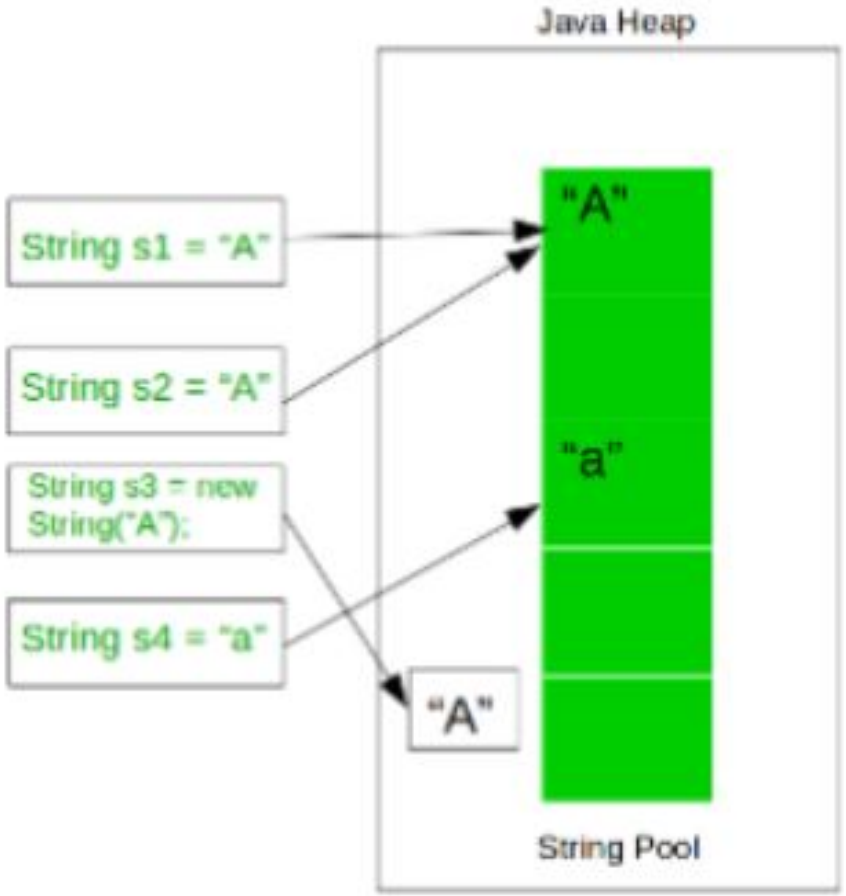
# Sobrescritura del método equals()



```java
public class Persona {
    private String nombre;
    private int edad;

    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true; // Mismo objeto
        Persona persona = (Persona) obj;
        // Comparar atributos
        return edad == persona.edad && nombre.equals(persona.nombre);
    }
}
```

# Comparar cadenas en Java






s1.equals(s2) → true
s1.equals(s3) → true
s1.equals(s4) → false
s1 == s2 → true
s1 == s3 → false
s1 == s4 → false

# Buenas prácticas al sobrescribir equals()



1. Consistencia con hashCode(): Siempre que sobrescribas `equals()`, también sobrescribe `hashCode()` para garantizar un comportamiento consistente.
2. Null Safety: Asegúrate de manejar valores nulos para evitar excepciones.
3. Simetría: Si `x.equals(y)` es `true`, entonces `y.equals(x)` también debe serlo.
4. Transitividad: Si `x.equals(y)` y `y.equals(z)` son `true`, entonces `x.equals(z)` también debe serlo.

# getClass()



Devuelve el **tipo del objeto en tiempo de ejecución**

```
Persona persona = new Persona();
Object personaObjeto = new Persona();

persona.getClass().getName()          → Persona
personaObjeto.getClass().getName()    → Persona

System.out.println(personaObjeto instanceof Persona);  → true
System.out.println(personaObjeto instanceof Object);   → true
```

# Ejercicio 11

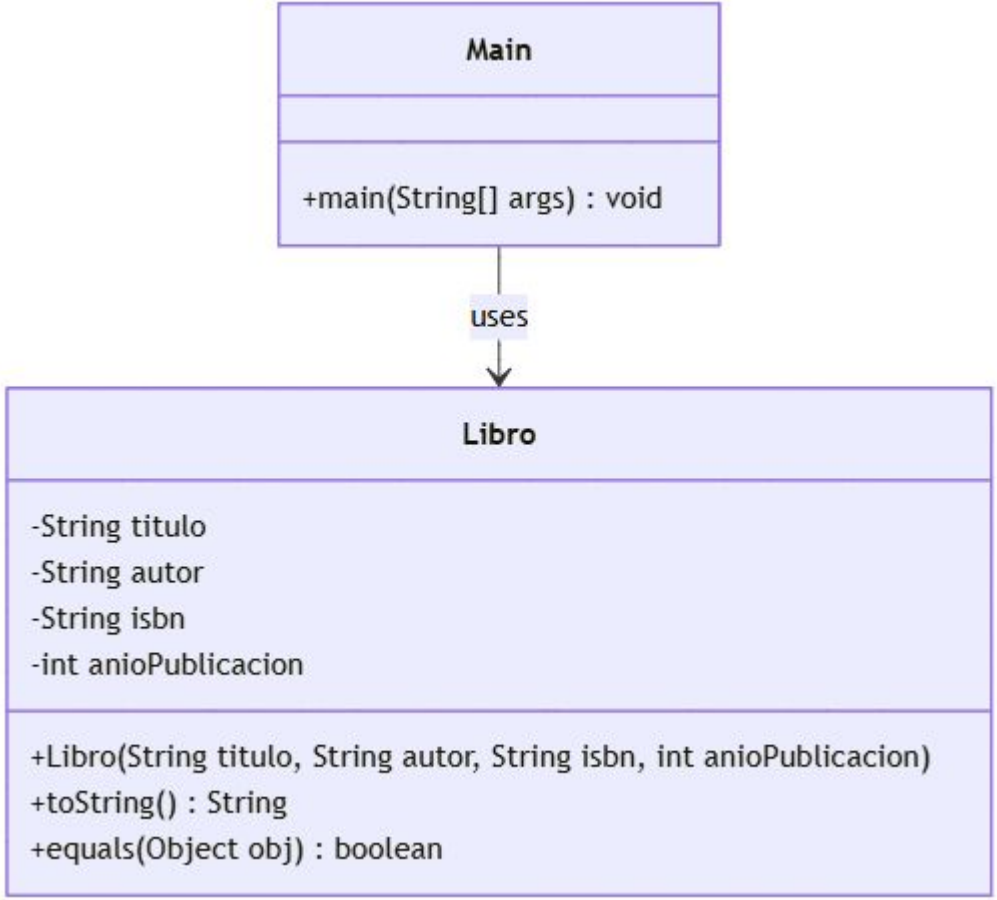


Crea una clase `Libro` con

- `titulo` (String)
- `autor` (String)
- `isbn` (String)
- `anioPublicacion` (

Tareas:

1. Implementa el con
2. Sobrescribe el mét
   "Título: [titulo], Aut
3. Sobrescribe el mét
4. En el método `main`
   - Imprime cac
   - Compara dc
   - Compara dc




con el siguiente formato:

ISBN.

# Responde…



## How to participate?

1. Go to wooclap.com
2. Enter the event code in the top banner

Event code
**RGDLUO**

Enable answers by SMS

# Métodos abstractos



Son métodos pensados solamente para ser sobreescritos

- Declarados sin implementación
- Utilizan la palabra clave `abstract`
- No tienen cuerpo de método, terminan con punto y coma
- Ejemplo: `public abstract void hacerSonido();`
- Los métodos abstractos deben ser implementados por las subclases concretas

# Métodos abstractos – Ejemplo



```java
abstract class Figura {
    // Método abstracto
    public abstract double calcularArea();

    // Método concreto
    public void mostrarTipo() {
        System.out.println("Soy una figura geométrica.");
    }
}

class Circulo extends Figura {
    private double radio;

    @Override
    public double calcularArea() {
        return Math.PI * radio * radio;
    }
}
```

# Clases abstractas



**Una clase abstracta es una clase que no puede ser instanciada directamente**

- Se declara usando la palabra clave `abstract`
- Una clase que contiene al menos un método abstracto debe ser declarada como abstracta
- Las clases abstractas pueden contener métodos abstractos y concretos (con implementación)
- Las subclases deben implementar todos los métodos abstractos

¿Para qué sirven?

- Sirven como base para otras clases
- Declaran el “contrato” que las subclases deben cumplir
- Facilita el polimorfismo y la abstracción

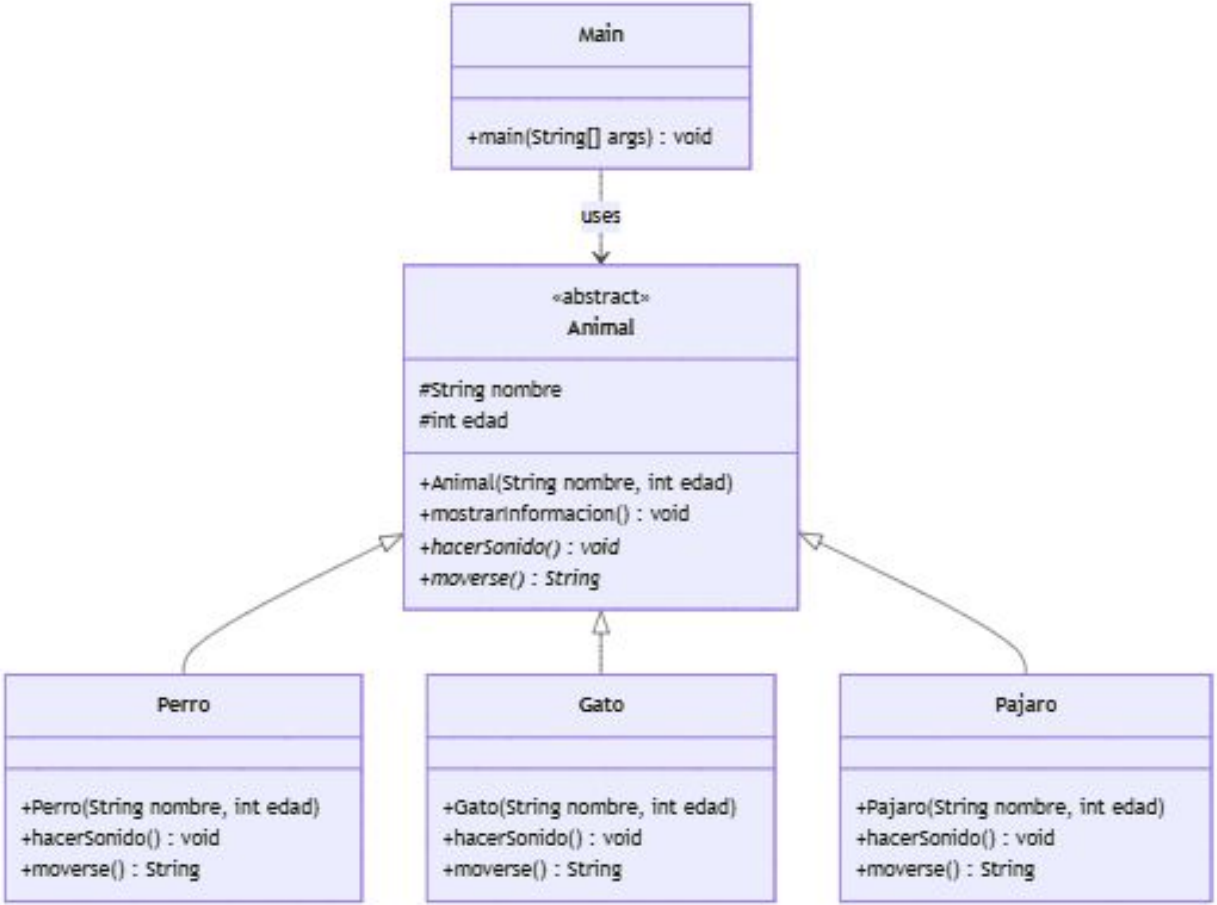
# Clases abstractas – Ejemplo



```java
abstract class Animal {
    protected String nombre;

    public abstract void hacerSonido();

    public void dormir() {
        System.out.println(nombre + " está durmiendo.");
    }
}

class Perro extends Animal {
    public void hacerSonido() {
        System.out.println("Guau!");
    }
}
```

# Ejercicio 12





# Un, dos, tres, responde otra vez



## How to participate?

1. Go to wooclap.com
2. Enter the event code in the top banner

Event code
**RGDLUO**

Enable answers by SMS

# Atributos y métodos finales



**Atributos Finales:**

- Son constantes; su valor no puede cambiarse después de la inicialización.
- Deben inicializarse al declararse o en el constructor.
- Ejemplo: `final int MAX_VALUE = 100;`

**Métodos Finales:**

- No pueden ser sobrescritos por subclases.
- Garantizan que la implementación permanezca inalterada.
- Ejemplo:

```java
public final void display() {
    System.out.println("Este es un método final.");
}
```

# Clases finales



- Una clase declarada como `final` no puede ser heredada por otras clases.
- Se utiliza la palabra clave `final` antes de la declaración de la clase.
- Todos los métodos de una clase final son implícitamente finales.
- Ejemplo:

```
public final class MiClaseFinal {
    // Contenido de la clase
}
```

<u>Usos comunes</u>:

- Clases de utilidad (como `java.lang.Math`)
- Clases inmutables (como `java.lang.String`)
- Clases que no deben ser extendidas por razones de seguridad o diseño →evitamos que nadie la modifique.

# Variables finales de tipo objeto



```
final Persona p = new Persona("Paco");
```

¿Puedo hacer cambios en el objeto p?

```
p.nombre = "Fran";
```

¿Puedo cambiar la referencia de p?

```
p = new Persona("Fran");
```

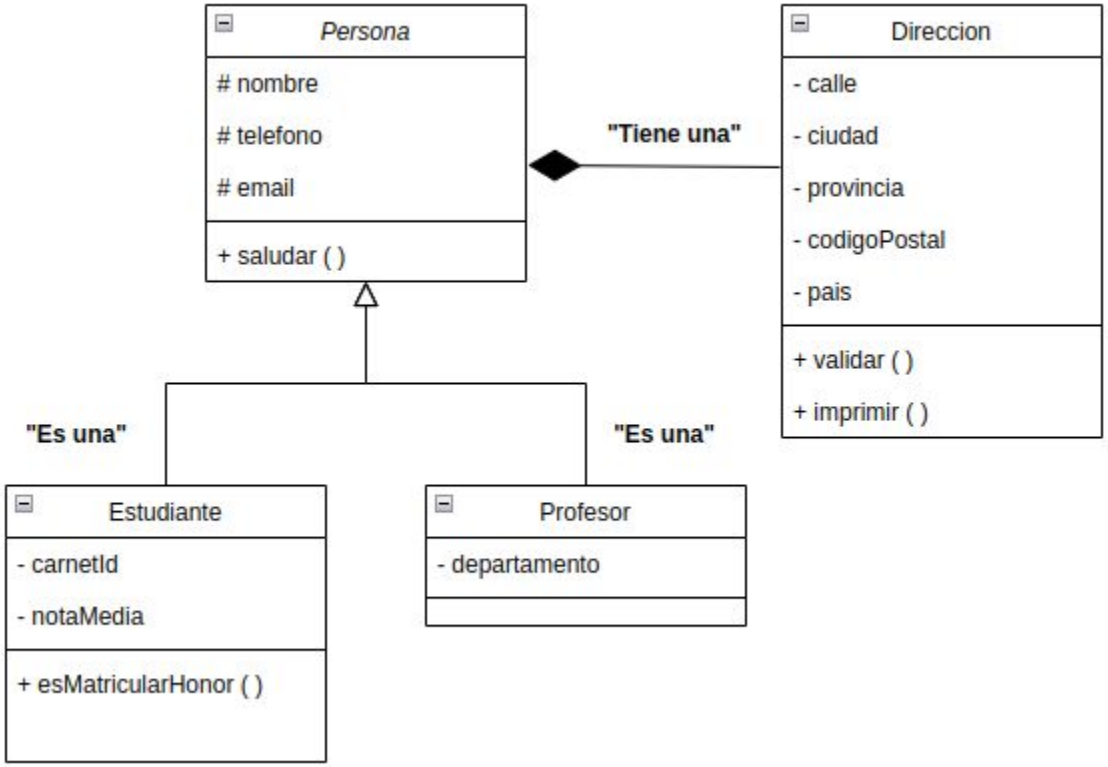
# Ejercicio 13



Creación de una Clase de Utilidades Matemáticas

# Ejercicio 14: COMPOSICIÓN Y HERENCIA






```java
public class Persona {
    private String nombre;
    private String telefono;
    private String email;

    private Direccion direccion;
```

```java
public class Estudiante extends Persona{
    private Integer carnet;
    private Float notaMedia;
```

# Paso por valor y por referencia



```
Persona p1 = new Persona();
p1.setNombre("Fran");
```

```
public void setNombre(String nombre) {   nombre: "Fran"
    this.nombre = nombre;   nombre: "Fran"   nombre: null
}
```

```
Direccion d1 = new Direccion();   d1: Direccion@697
d1.setCalle("13 Rue del Percebe");
```

```
p1.setDireccion(d1);   p1: Persona@692   d1: Direccion@697
```

```
direccion = {Direccion@697}
> calle = "13 Rue del Percebe"
  ciudad = null
  provincia = null
  codigoPostal = null
  pais = null
```

```
public void setDireccion(Direccion direccion) {   direccion: Direccion@697
    this.direccion = direccion;   direccion: Direccion@697   direccion: null
}
```

# Paso por valor y por referencia



- Los argumentos se pueden pasar de dos formas:
  - Por valor → se pasa una copia del original (cambios en el parámetro **no afectan** al original)
  - Por referencia → se pasa la referencia (dirección de memoria) del original (cambios en el parámetro **sí afectan** al original)

# Paso por valor y por referencia



- En Java siempre se pasan los argumentos por valor, pero…
  - Argumentos de tipo primitivo → Se pasa la copia del dato
  - Argumentos de tipo objeto → Se pasa la copia de la referencia

Por tanto, cuando se pasa la referencia, ¡ojo!:
**las modificaciones sobre la referencia modifican su valor original**

# Ejercicio 15



Implementa y ejecuta la siguiente clase y comprueba los resultados obtenidos con los que esperabas.

```java
public class ComparacionParametros {

    public static void modificarPrimitivo(int numero) {
        numero = numero * 2;
        System.out.println("Dentro del método, numero = " + numero);
    }

    public static void modificarObjeto(StringBuilder texto) {
        texto.append(" modificado");
        System.out.println("Dentro del método, texto = " + texto);
    }

    public static void main(String[] args) {
        int valorPrimitivo = 5;
        StringBuilder valorObjeto = new StringBuilder("Hola");

        System.out.println("Antes de llamar al método, valorPrimitivo = " + valorPrimitivo);
        modificarPrimitivo(valorPrimitivo);
        System.out.println("Después de llamar al método, valorPrimitivo = " + valorPrimitivo);

        System.out.println("\nAntes de llamar al método, valorObjeto = " + valorObjeto);
        modificarObjeto(valorObjeto);
        System.out.println("Después de llamar al método, valorObjeto = " + valorObjeto);
    }
}
```

# Recursos y referencias



- Libro que tenéis en la biblioteca y en el departamento
- Diapositivas en Moodle
- Glosario en Moodle
- Ejercicios de refuerzo y de ampliación
- Vuestro portfolio